

Computational Thinking with Python

Session 2 – Flow Control

STUDENTS WILL MASTER FLOW CONTROL TO CREATE DYNAMIC PROGRAMS

At the end of this session, you should be able to comprehend, trace, adapt and create Python code that:

- Uses **Boolean conditions** to evaluate expressions
- Uses **selection** with `if`, `elif`, and `else`
- Handles one, two, or multiple outcomes with selection
- **Iterates** using `while` loops (indefinite iteration)
- **Iterates** using `for` loops (definite iteration)
- Uses `break` and `continue` to control loop execution

AGENDA

- 1 Boolean Conditions
- 2 Selection
- 3 Iteration

FLOW CONTROL ENABLES PROGRAMS TO MAKE DECISIONS AND REPEAT ACTIONS

So far:

Your programs run line by line, from top to bottom.

But real programs need to:

- **Make decisions** – Should a customer get a discount?
- **Repeat actions** – Process all items in a cart
- **React to conditions** – Show different messages

Flow control lets programs:

- Execute code **conditionally**
- **Loop** through data
- **Adapt** to different inputs

Flow control makes programs smart!

AGENDA

- 1 Boolean Conditions**
- 2 Selection
- 3 Iteration

BOOLEAN OPERATORS COMPARE VALUES AND RETURN TRUE OR FALSE

- Python uses Boolean variables to **evaluate conditions**
- The Boolean values `True` and `False` are returned when an expression is evaluated

Operator	Meaning
<code>==</code>	Equal to / The same as
<code>!=</code>	Not equal to
<code>></code>	Greater than
<code>>=</code>	Greater than or equal to
<code><</code>	Less than
<code><=</code>	Less than or equal to

BOOLEAN OPERATORS COMPARE TWO PIECES OF DATA

data1 *boolean operator* data2

The result is always True or False!

CONDITIONS EVALUATE TO TRUE OR FALSE BASED ON THE COMPARISON

Number comparisons:

```
5 == 5  # True
5 != 5  # False
6 < 4   # False
4 >= 4  # True
4 > 4   # False
```

String comparisons:

```
"Giraffe" == "Giraffe" # True
"Giraffe" == "giraffe" # False
"Giraffe" != "Hippo"   # True
```

Important: String comparisons are case-sensitive!

LOGICAL OPERATORS COMBINE MULTIPLE CONDITIONS

The operators **and**, **or**, and **not** build complex expressions:

Operator	What it does	Examples
and	True if both are true	True and False $\rightarrow$ False True and True $\rightarrow$ True
or	True if one is true	True or False $\rightarrow$ True False or False $\rightarrow$ False
not	Negates the statement	not True $\rightarrow$ False not False $\rightarrow$ True

APPLY BOOLEAN CONDITIONS IN PRIMM EXERCISES

Practice: Open `01_Boolean_Conditions` in your Codespace

Complete: `01_predict.py`, `02_predict.py`, `03_investigate.py`, `04_modify.py`,
`05_make.py`

Predict → Investigate → Modify → Make

AGENDA

- 1 Boolean Conditions
- 2 Selection**
- 3 Iteration

THE IF STATEMENT EXECUTES CODE ONLY WHEN A CONDITION IS TRUE

if boolean condition:

Do this

- `if` – Starts the selection statement
- `boolean condition` – Expression that evaluates to `True` or `False`
- `:` – Colon is required after the condition
- Indented code – Code must be indented (4 spaces)

SELECTION WITH ONE OUTCOME RUNS CODE ONLY WHEN TRUE

How it works:

1. Check the condition
2. If True: run the indented code
3. If False: skip the indented code
4. Continue with code after the if block

Note: Code after the if block always runs!

Example:

```
num1 = 10  
if num1 == 10:  
    print("num1 is ten!")  
print("Program continues...")
```

```
num1 is ten!  
Program continues...
```

THE ELSE CLAUSE HANDLES THE FALSE CASE

Without else:

```
num1 = 100
num2 = 50

if num1 > num2:
    print("num1 is bigger")
print("End of program")
```

What if we want different behavior when False?

With else:

```
num1 = 100
num2 = 50

if num1 > num2:
    print("num1 is bigger")
else:
    print("num2 is bigger")
print("End of program")
```

ONLY ONE BRANCH EXECUTES IN IF/ELSE CHAINS

The flow:

1. Check the condition
2. If True: run the if block
3. If False: run the else block
4. Continue after the selection

Only one branch executes!

Key points:

- else has no condition
- else still needs a colon
- Code under else must be indented

```
if num1 == 42:  
    print("Meaning of life!")  
else:  
    print("Keep searching...")
```

TWO OUTCOMES ARE NOT ALWAYS ENOUGH FOR REAL DECISIONS

```
if weather == "rain":  
    print("Take your umbrella")  
else:  
    print("No special advice for you")
```

What if we want to handle more weather conditions?

THE ELIF KEYWORD ADDS ADDITIONAL CONDITIONS BETWEEN IF AND ELSE

Add `elif` with a condition between `if` and `else`.

You can add as many `elif`s as you need.

The algorithm:

1. Start with an `if` – set the first condition
2. Add `elif`s as needed – each with a new condition
3. Finish with an `else` – no condition (catches everything)

MULTIPLE CONDITIONS ARE CHECKED IN ORDER FROM TOP TO BOTTOM

Three outcomes:

```
if weather == "rain":  
    print("Take umbrella")  
elif weather == "snow":  
    print("Take scarf")  
else:  
    print("No special advice")
```

Four outcomes:

```
if weather == "rain":  
    print("Take umbrella")  
elif weather == "snow":  
    print("Take scarf")  
elif weather == "sunny":  
    print("Take sunglasses")  
else:  
    print("No advice")
```

ONLY THE FIRST MATCHING BRANCH EXECUTES IN IF/ELIF/ELSE

```
if num1 > num2:
    print(str(num1) + " is bigger.")
elif num1 < num2:
    print(str(num2) + " is bigger.")
else:
    print("The numbers are the same")
```

Remember:

- if and elif both need conditions
- else has no condition – it's the “catch all”
- Conditions are checked in order from top to bottom

PRACTICE SELECTION WITH HANDS-ON EXERCISES

Practice: Open `02_Selection` in your Codespace

Complete: `01_predict.py`, `02_predict.py`, `03_investigate.py`, `04_modify.py`,
`05_modify.py`, `06_make.py`

Predict → Investigate → Modify → Make

AGENDA

- 1 Boolean Conditions
- 2 Selection
- 3 Iteration**

ITERATION REPEATS A BLOCK OF CODE MULTIPLE TIMES

Indefinite loops (while):

- Run until a condition becomes False
- Number of iterations unknown beforehand
- Can be infinite (run forever!)

Definite loops (for):

- Iterate through a sequence/collection
- Execute an exact number of times
- Number of iterations known beforehand

Key Takeaway: Both are essential tools!

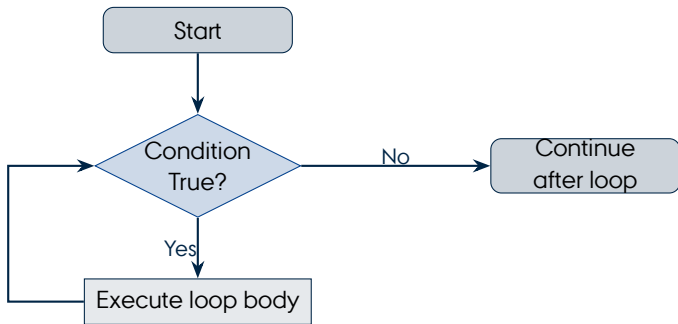
THE WHILE LOOP REPEATS CODE AS LONG AS A CONDITION IS TRUE

`while boolean condition:`

Do this

- `while` – Starts the iteration statement
- `boolean condition` – Loop continues while this is True
- `:` – Colon is required after the condition
- The indented code repeats while the condition remains True

WHILE LOOPS CHECK THE CONDITION BEFORE EACH ITERATION



Key: Something inside the loop body must change the condition!

THIS COUNTDOWN SHOWS HOW WHILE LOOPS WORK

Example:

```
n = 5
while n > 0:
    print(n)
    n = n - 1
print("Blastoff!")
```

Step-by-step:

1. $n = 5$ – Initialize
2. Check: $5 > 0$? Yes $\rightarrow$ print 5, n becomes 4
3. Check: $4 > 0$? Yes $\rightarrow$ print 4, n becomes 3
4. ... continues ...
5. Check: $0 > 0$? No $\rightarrow$ exit loop
6. Print "Blastoff!"

ITERATION VARIABLES CONTROL HOW MANY TIMES A LOOP RUNS

```
n = 5      # Initialize the iteration variable
while n > 0: # Condition uses the iteration variable
    print(n)
    n = n - 1 # Update the iteration variable
print("Blastoff!")
```

Iteration variables must be:

- Initialized before the loop
- Checked in the condition
- Changed each time through the loop
- Eventually make the condition False

FORGETTING TO UPDATE THE ITERATION VARIABLE CAUSES INFINITE LOOPS

What's wrong here?

```
n = 5
while n > 0:
    print(n)
print("Completed")
```

The variable `n` is never changed!

The problem:

- The condition `n > 0` will always be True
- `n` stays at 5 forever
- This loop will print 5 forever!

This is an infinite loop!

THE BREAK STATEMENT EXITS A LOOP IMMEDIATELY

Example:

```
while True:
    line = input("Enter: ")
    if line == "done":
        break
    print(line)
print("Completed")
```

How it works:

- `while True:` creates an “infinite” loop
- `break` immediately exits the loop
- Program continues after the loop

Use case: Exit a loop based on something that happens inside the loop.

THE CONTINUE STATEMENT SKIPS TO THE NEXT ITERATION

Example:

```
while True:
    line = input("Enter: ")
    if line == "skip":
        continue
    if line == "done":
        break
    print(line)
```

How it works:

- `continue` skips the rest of the iteration
- Loop goes back to check the condition
- Useful for skipping certain values

Difference:

- `break` – exits the entire loop
- `continue` – skips to next iteration

THE FOR LOOP ITERATES THROUGH EACH ITEM IN A SEQUENCE

`for i in sequence:`

`Do this`

- `for` – Starts the iteration statement
- `i` – The iteration variable (takes on each value)
- `in` – Keyword connecting variable to sequence
- `sequence` – A list, string, range, or other iterable
- The indented code repeats for each item in the sequence

FOR LOOPS AUTOMATICALLY ITERATE THROUGH EACH ITEM

Example:

```
for i in [5, 4, 3, 2, 1]:  
    print(i)  
print("Blastoff!")
```

The iteration variable `i` takes on each value in the list, one at a time.

Output:

```
5  
4  
3  
2  
1  
Blastoff!
```

THE RANGE() FUNCTION GENERATES SEQUENCES OF NUMBERS

Examples:

```
for i in range(5):  
    print(i)  
# Output: 0, 1, 2, 3, 4  
  
for i in range(1, 6):  
    print(i)  
# Output: 1, 2, 3, 4, 5
```

range() variations:

- `range(n)` – 0 to $n-1$
- `range(start, stop)` – start to $stop-1$
- `range(start, stop, step)` – with step

```
for i in range(0, 10, 2):  
    print(i)  
# Output: 0, 2, 4, 6, 8
```

THE ENUMERATE() FUNCTION PROVIDES BOTH INDEX AND VALUE

Example:

```
fruits = ["apple", "banana",  
          "cherry"]  
  
for i, f in enumerate(fruits):  
    print(f"{i}: {f}")
```

Output:

```
0: apple  
1: banana  
2: cherry
```

`enumerate()` returns pairs of (index, value)
– very useful!

LOOPS ARE ESSENTIAL FOR CALCULATING TOTALS FROM LISTS

Example: Shopping cart total

```
prices = [19.99, 5.50, 12.00,
          8.25]
total = 0
for price in prices:
    total = total + price
print(f"Total: {total} EUR")
```

Output:

Total: 45.74 EUR

The pattern:

1. Initialize an accumulator (total = 0)
2. Loop through each item
3. Add each item to the accumulator

CHOOSE WHILE FOR UNKNOWN ITERATIONS, FOR FOR KNOWN ITERATIONS

Use while when:

- You don't know how many iterations
- Loop depends on a condition
- User input controls the loop
- You need an “infinite” loop with `break`

Use for when:

- You know exactly how many iterations
- Iterating through a collection
- Processing each item in a list
- Counting through a range

Key Takeaway: `while` = indefinite (condition-based); `for` = definite (sequence-based)

PRACTICE ITERATION WITH HANDS-ON EXERCISES

Practice: Open 03_Iteration in your Codespace

Complete: 01_predict.py, 02_predict.py, 03_investigate.py, 04_modify.py,
05_modify.py, 06_make.py

Predict → Investigate → Modify → Make

FLOW CONTROL MAKES PROGRAMS DYNAMIC AND RESPONSIVE

- ✓ **Boolean conditions** evaluate to True or False
 - Comparison operators: `==`, `!=`, `<`, `>`, `<=`, `>=`
 - Logical operators: `and`, `or`, `not`
- ✓ **Selection** controls which code runs
 - `if` – one outcome
 - `if/else` – two outcomes
 - `if/elif/else` – multiple outcomes
- ✓ **Iteration** repeats code
 - `while` – indefinite loops
 - `for` – definite loops
 - `break` exits; `continue` skips

Thank You!

Nikolai Stein

